



객체 포인터와 객체 배열, 객체의 동적 생성

학습 목표

1. 객체에 대한 포인터를 선언하고 활용할 수 있다.
2. 객체의 배열을 선언하고 활용할 수 있다.
3. new를 이용하여 동적으로 메모리나 배열을 할당 받고 delete를 이용하여 반환할 수 있다.
4. new를 이용하여 동적으로 객체나 객체 배열을 할당 받고 delete를 이용하여 반환할 수 있다.
5. this 포인터의 개념을 이해하고, 활용할 수 있다.
6. string 클래스를 이용하여 문자열을 다룰 수 있다.

객체 포인터

3

- 객체에 대한 포인터
 - ▣ C 언어의 포인터와 동일
 - ▣ 객체의 주소 값을 가지는 변수
- 포인터로 멤버를 접근할 때
 - ▣ 객체포인터->멤버

```
class Circle {  
    int radius;  
public:  
    Circle() {radius = 1;}  
    Circle(int r) { radius = r; }  
    double getArea();  
};
```

```
Circle donut;  
double d = donut.getArea();  
  
Circle *p; // (1)  
p = &donut; // (2)  
d = p->getArea(); // (3)
```

객체에 대한 포인터 선언

포인터에 객체 주소 저장

멤버 함수 호출

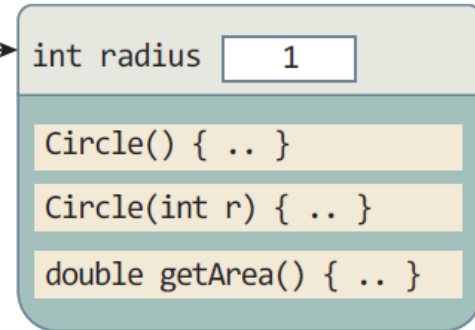
(1) Circle *p;



(2) p=&donut;



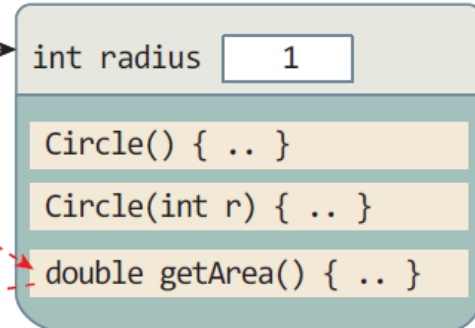
donut 객체



(3) d=p->getArea();



donut 객체



호출



예제 4-1 객체 포인터 선언 및 활용

4

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle() { radius = 1; }
    Circle(int r) { radius = r; }
    double getArea();
};

double Circle::getArea() {
    return 3.14*radius*radius;
}
```

```
int main() {
    Circle donut; // donut()으로 선언하면 안됨 (함수선언으로 인식함)
    Circle pizza(30);

    // 객체 이름으로 멤버 접근
    cout << donut.getArea() << endl;

    // 객체 포인터로 멤버 접근
    Circle *p;
    p = &donut;
    cout << p->getArea() << endl; // donut의 getArea() 호출
    cout << (*p).getArea() << endl; // donut의 getArea() 호출

    p = &pizza;
    cout << p->getArea() << endl; // pizza의 getArea() 호출
    cout << (*p).getArea() << endl; // pizza의 getArea() 호출
}
```

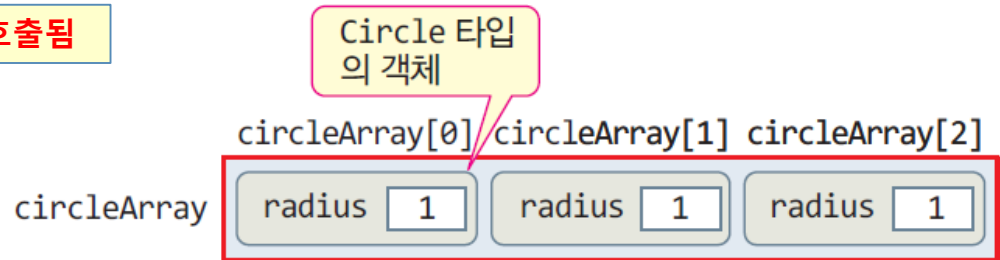
```
3.14
3.14
3.14
2826
2826
```


배열 생성과 활용(예제 4-2의 실행 과정)

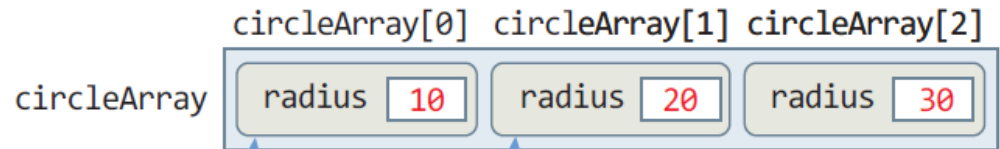
5

배열 원소인 각 객체는 기본 생성자가 자동 호출됨

(1) `Circle circleArray[3];`



(2) `circleArray[0].setRadius(10);`
`circleArray[1].setRadius(20);`
`circleArray[2].setRadius(30);`



(3) `Circle *p;`



(4) `p = circleArray;`



(5) `p++;`



```
class Circle {  
    int radius;  
public:  
    Circle() { radius = 1; }  
    Circle(int r) { radius = r; }  
    void setRadius(int r) { radius = r; }  
    double getArea();  
};
```

예제 4- 2 Circle 클래스의 배열 선언 및 활용

6

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle() { radius = 1; }
    Circle(int r) { radius = r; }
    void setRadius(int r) { radius = r; }
    double getArea();
};

double Circle::getArea() {
    return 3.14*radius*radius;
}
```

circleArray[0] circleArray[1] circleArray[2]

radius	10	radius	20	radius	30
--------	----	--------	----	--------	----



```
int main() {
    Circle circleArray[3]; // (1) Circle 객체 배열 생성

    // 배열의 각 원소 객체의 멤버 접근
    circleArray[0].setRadius(10); // (2)
    circleArray[1].setRadius(20);
    circleArray[2].setRadius(30);

    for(int i=0; i<3; i++) // 배열의 각 원소 객체의 멤버 접근
        cout << "Circle " << i << "의 면적은 " << circleArray[i].getArea() << endl;

    Circle *p; // (3)
    p = circleArray; // (4)
    for(int i=0; i<3; i++) { // 객체 포인터로 배열 접근
        cout << "Circle " << i << "의 면적은 " << p->getArea() << endl;
        p++; // (5)
    }
    // 함수가 리턴될 때 배열의 각 원소의 소멸자가 호출됨(생성의 역순으로 )
}
```

Circle 0의 면적은 314
Circle 1의 면적은 1256
Circle 2의 면적은 2826
Circle 0의 면적은 314
Circle 1의 면적은 1256
Circle 2의 면적은 2826

객체 배열의 생성 및 소멸

7

□ 객체 배열 선언 가능

▣ 기본 타입 배열 선언과 형식 동일

- `int n[3];` // 정수형 배열 선언
- `Circle c[3];` // Circle 타입의 배열 선언

□ 객체 배열 선언

1. 객체 배열을 위한 공간 할당

2. 배열의 각 원소 객체마다 생성자 실행

- `c[0]`의 생성자, `c[1]`의 생성자, `c[2]`의 생성자 실행
- 매개 변수 없는 생성자 호출

▣ 매개 변수 있는 생성자를 호출할 수 없음

- `Circle circleArray[3](5);` // 오류

□ 배열 소멸

▣ 배열의 각 객체마다 소멸자 호출. 생성의 반대순으로 소멸

- `c[2]`의 소멸자, `c[1]`의 소멸자, `c[0]`의 소멸자 실행

객체 배열 생성시 기본 생성자 호출

8

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    double getArea() {
        return 3.14*radius*radius;
    }
};

int main() {
    Circle circleArray[3];
}
```

컴파일러가 자동으로 기본 생성자
Circle() {} 삽입.
컴파일 오류가 발생하지 않음

기본 생성자 Circle() 호출

(a) 생성자가 선언되어
있지 않은 Circle 클래스

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle(int r) { radius = r; }
    double getArea() {
        return 3.14*radius*radius;
    }
};

int main() {
    Circle waffle(15);
    Circle circleArray[3];
}
```

Circle(int r)
호출

기본 생성자 Circle() 호출.
기본 생성자가 없으므로 컴
파일 오류

error.cpp(15): error C2512: 'Circle' : 사용할
수 있는 적절한 기본 생성자가 없습니다

(b) 기본 생성자가 없으므로 컴파일 오류

객체 배열 초기화

9

□ 객체 배열 초기화 방법

- ▣ 배열의 각 원소 객체당 생성자 지정하는 방법

```
Circle circleArray[3] = { Circle(10), Circle(20), Circle() };
```

- circleArray[0] 객체가 생성될 때, 생성자 Circle(10) 호출
- circleArray[1] 객체가 생성될 때, 생성자 Circle(20) 호출
- circleArray[2] 객체가 생성될 때, 생성자 Circle() 호출

```
class Circle {  
    int radius;  
public:  
    Circle() { radius = 1; }  
    Circle(int r) { radius = r; }  
    void setRadius(int r) { radius = r; }  
    double getArea();  
};
```

예제 4-3 객체 배열 초기화

10

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle() { radius = 1; }
    Circle(int r) { radius = r; }
    void setRadius(int r) { radius = r; }
    double getArea();
};

double Circle::getArea() {
    return 3.14*radius*radius;
}

int main() {
    Circle circleArray[3] = { Circle(10), Circle(20), Circle() }; // Circle 배열 초기화

    for(int i=0; i<3; i++)
        cout << "Circle " << i << "의 면적은 " << circleArray[i].getArea() << endl;
}
```

circleArray[0] 객체가 생성될 때, 생성자 Circle(10),
circleArray[1] 객체가 생성될 때, 생성자 Circle(20),
circleArray[2] 객체가 생성될 때, 기본 생성자 Circle()
이 호출된다.

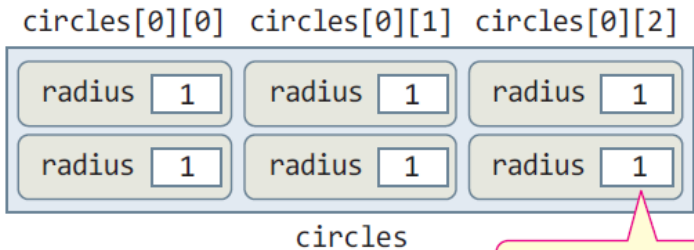
Circle 0의 면적은 314
Circle 1의 면적은 1256
Circle 2의 면적은 3.14

객체의 2차원 배열

11

Circle() 호출

```
Circle circles[2][3];
```

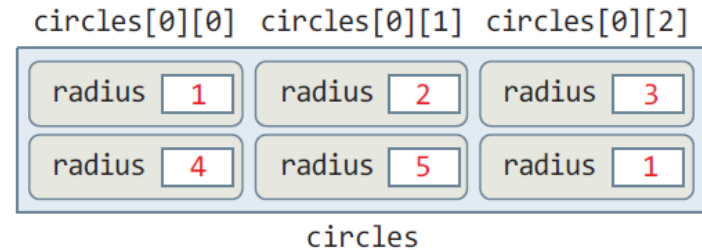


(a) 2차원 배열 선언 시

Circle(int r) 호출

```
Circle circles[2][3] = { { Circle(1), Circle(2), Circle(3) },  
                          { Circle(4), Circle(5), Circle() } };
```

Circle() 호출



(b) 2차원 배열 선언과 초기화

```
circles[0][0].setRadius(1);  
circles[0][1].setRadius(2);  
circles[0][2].setRadius(3);  
circles[1][0].setRadius(4);  
circles[1][1].setRadius(5);  
circles[1][2].setRadius(6);
```

2차원 배열을 초기화하는 다른 방식

```
class Circle {  
    int radius;  
public:  
    Circle() { radius = 1; }  
    Circle(int r) { radius = r; }  
    void setRadius(int r) { radius = r; }  
    double getArea();  
};
```

예제 4-4 Circle 클래스의 2차원 배열 선언 및 활용

12

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle() { radius = 1; }
    Circle(int r) { radius = r; }
    void setRadius(int r) { radius = r; }
    double getArea();
};

double Circle::getArea() {
    return 3.14*radius*radius;
}
```

```
int main() {
    Circle circles[2][3];
```

```
    circles[0][0].setRadius(1);
```

```
    circles[0][1].setRadius(2);
```

```
    circles[0][2].setRadius(3);
```

```
    circles[1][0].setRadius(4);
```

```
    circles[1][1].setRadius(5);
```

```
    circles[1][2].setRadius(6);
```

Circle circles[2][3] =
{ { Circle(1), Circle(2), Circle(3) },
 { Circle(4), Circle(5), Circle() } };

```
    for(int i=0; i<2; i++) // 배열의 각 원소 객체의 멤버 접근
        for(int j=0; j<3; j++) {
            cout << "Circle [" << i << ", " << j << "]의 면적은 ";
            cout << circles[i][j].getArea() << endl;
        }
}
```

circles[0][0] circles[0][1] circles[0][2]

radius 1	radius 2	radius 3
radius 4	radius 5	radius 6

Circle [0,0]의 면적은 3.14
Circle [0,1]의 면적은 12.56
Circle [0,2]의 면적은 28.26
Circle [1,0]의 면적은 50.24
Circle [1,1]의 면적은 78.5
Circle [1,2]의 면적은 113.04

동적 메모리 할당 및 반환

13

- 정적 할당
 - ▣ 변수 선언을 통해 필요한 메모리 할당
 - 많은 양의 메모리는 배열 선언을 통해 할당
- 동적 할당
 - ▣ 필요한 양이 예측되지 않는 경우. 프로그램 작성시 할당 받을 수 없음
 - ▣ 실행 중에 운영체제로부터 할당 받음
 - 힙(heap)으로부터 할당
 - 힙은 운영체제가 소유하고 관리하는 메모리. 모든 프로세스가 공유할 수 있는 메모리
- C 언어의 동적 메모리 할당 : malloc()/free() 라이브러리 함수 사용
- C++의 동적 메모리 할당/반환
 - ▣ new 연산자
 - 기본 타입 메모리 할당, 배열 할당, 객체 할당, 객체 배열 할당
 - 객체의 동적 생성 - 힙 메모리로부터 객체를 위한 메모리 할당 요청
 - 객체 할당 시 생성자 호출
 - ▣ delete 연산자
 - new로 할당 받은 메모리 반환
 - 객체의 동적 소멸 - 소멸자 호출 뒤 객체를 힙에 반환

new와 delete 연산자

14

- C++의 기본 연산자
- new/delete 연산자의 사용 형식

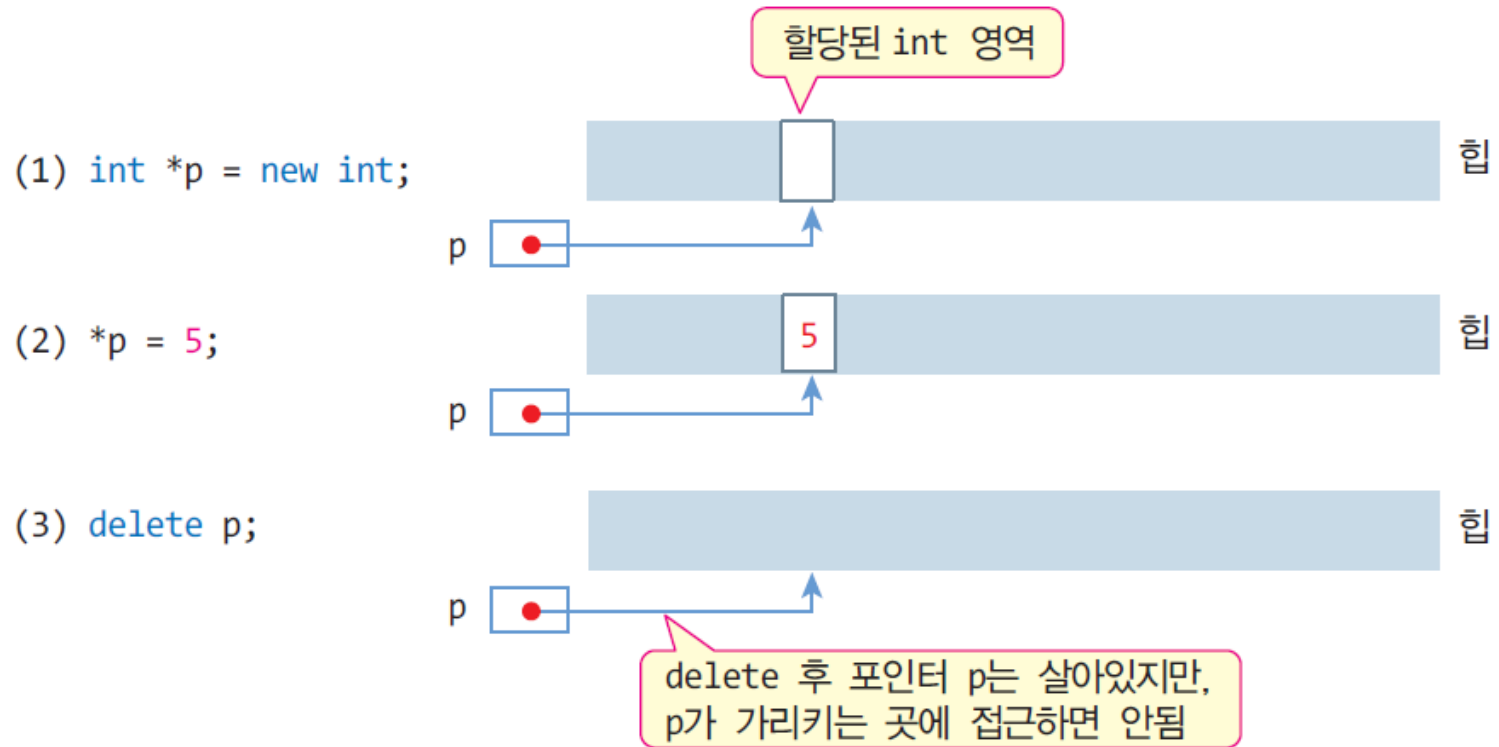
```
데이터타입 *포인터변수 = new 데이터타입 ;  
delete 포인터변수;
```

- new/delete의 사용

```
int    *pInt    = new int;    // int 타입의 메모리 동적 할당  
char   *pChar   = new char;   // char 타입의 메모리 동적 할당  
Circle *pCircle = new Circle(); // Circle 클래스 타입의 메모리 동적 할당  
  
delete pInt;           // 할당 받은 정수 공간 반환  
delete pChar;          // 할당 받은 문자 공간 반환  
delete pCircle;        // 할당 받은 객체 공간 반환
```


기본 타입의 메모리 동적 할당 및 반환

15



예제 4-5 정수형 공간의 동적 할당 및 반환 예

16

```
#include <iostream>
using namespace std;
```

```
int main() {
    int *p;
```

int 타입 1개 할당

```
    p = new int;
```

```
    if(!p) {
```

p 가 nullptr이면,
메모리 할당 실패

NULL도 가능하지만
C++에선 **nullptr** 사용

if (p == nullptr) 과 동일

```
        cout << "메모리를 할당할 수 없습니다.";
        return 0;
```

if (p) 는 if (p != nullptr)과 동일

```
    }
```

```
    *p = 5; // 할당 받은 정수 공간에 5 삽입
```

```
    int n = *p;
```

```
    cout << "*p = " << *p << '\n';
```

```
    cout << "n = " << n << '\n';
```

```
    delete p;
```

할당 받은 메모리 반환

```
}
```

```
*p = 5
n = 5
```

delete 사용 시 주의 사항

17

- 적절치 못한 포인터로 delete하면 실행 시간 오류 발생
 - ▣ 동적으로 할당 받지 않는 메모리 반환 - 오류

```
int n;  
int *p = &n;  
delete p; // 실행 시간 오류  
// 포인터 p가 가리키는 메모리는 동적으로 할당 받은 것이 아님
```

- ▣ 동일한 메모리 두 번 반환 - 오류

```
int *p = new int;  
delete p; // 정상적인 메모리 반환  
delete p; // 실행 시간 오류. 이미 반환한 메모리를 중복 반환할 수 없음
```

배열의 동적 할당 및 반환

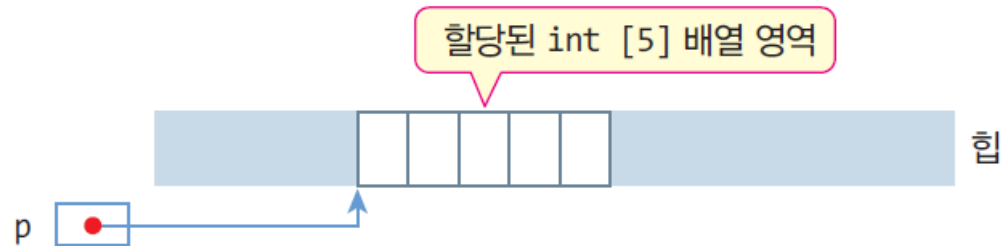
18

□ new/delete 연산자의 사용 형식

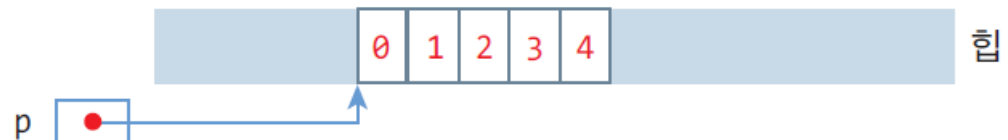
데이터타입 *포인터변수 = **new** 데이터타입 [배열의 크기]; // 동적 배열 할당
delete [] 포인터변수; // 배열 반환

int *p = new int;

(1) int *p = new int [5];

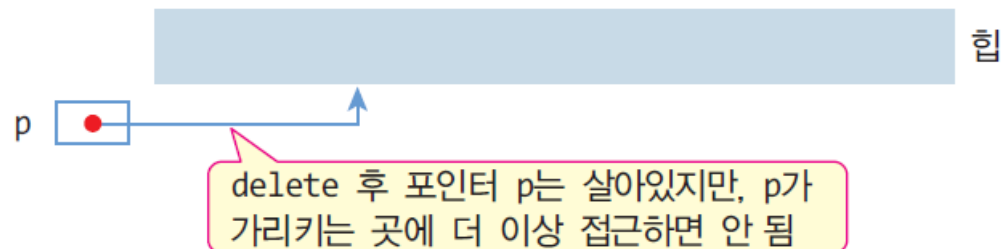


(2) for(int i=0; i<5; i++)
 p[i] = i;



(3) delete [] p;

delete p;



예제 4-6 정수형 배열의 동적 할당 및 반환

19

사용자로부터 입력할 정수의 개수를
입력 받아 배열을 동적 할당 받고,
하나씩 정수를 입력 받은 후
합을 출력하는 프로그램을 작성하라.

입력할 정수의 개수는?4

1번째 정수: 4

2번째 정수: 20

3번째 정수: -5

4번째 정수: 9

평균 = 7

```
for(int i=0; i<n; i++)  
    sum += *(p+i);
```

```
for(int i=0; i<n; i++, p++)  
    sum += *p;
```

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "입력할 정수의 개수는?";  
    int n;  
    cin >> n; // 정수의 개수 입력  
    if(n <= 0) return 0;  
    int *p = new int[n]; // n 개의 정수 배열 동적 할당  
    if(!p) {  
        cout << "메모리를 할당할 수 없습니다.";  
        return 0;  
    }  
  
    for(int i=0; i<n; i++) {  
        cout << i+1 << "번째 정수: "; // 프롬프트 출력  
        cin >> p[i]; // 키보드로부터 정수 입력  
    }  
  
    int sum = 0;  
    for(int i=0; i<n; i++)  
        sum += p[i];  
    cout << "평균 = " << sum/n << endl;  
  
    delete [] p; // 배열 메모리 반환  
}
```

동적 할당 메모리 초기화 및 delete 시 유의 사항

20

□ 동적 할당 메모리 초기화

▣ 동적 할당 시 초기화

```
데이터타입 *포인터변수 = new 데이터타입(초깃값);
```

```
int *pInt = new int(20); // 20으로 초기화된 int 타입 할당  
char *pChar = new char('a'); // 'a'로 초기화된 char 타입 할당
```

```
int *pInt = new int;  
char *pChar = new char;
```

▣ 배열의 동적 할당 시 초기화

```
int *pArray = new int [10](20);           // 구문 오류. 컴파일 오류 발생  
int *pArray = new int(20)[10];            // 구문 오류. 컴파일 오류 발생  
int *pArray = new int [] { 1, 2, 3, 4 }    // 1,2,3,4로 초기화된 정수 배열 생성
```

□ delete시 [] 생략

▣ 컴파일 오류는 아니지만 비정상적인 반환(실행시간 오류 발생)

```
int *p = new int [10];  
delete p; // 비정상 반환. delete [] p;로 하여야 함.
```

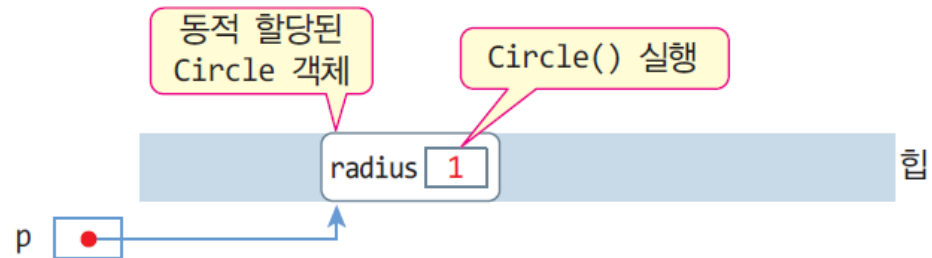
```
int *q = new int;  
delete [] q; // 비정상 반환. delete q;로 하여야 함.
```

객체의 동적 생성 및 반환

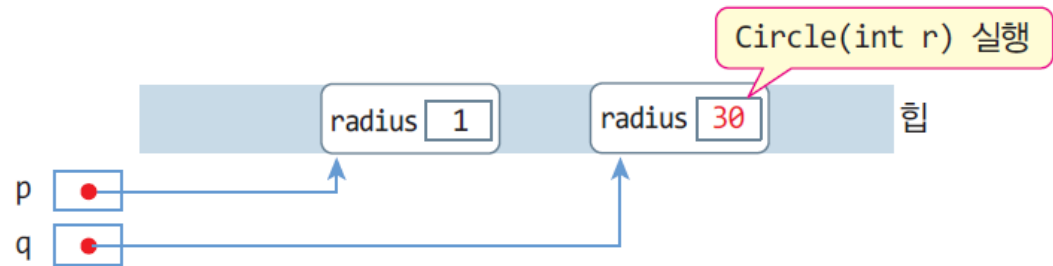
21

```
클래스이름 *포인터변수 = new 클래스이름;  
클래스이름 *포인터변수 = new 클래스이름(생성자매개변수리스트);  
delete 포인터변수;
```

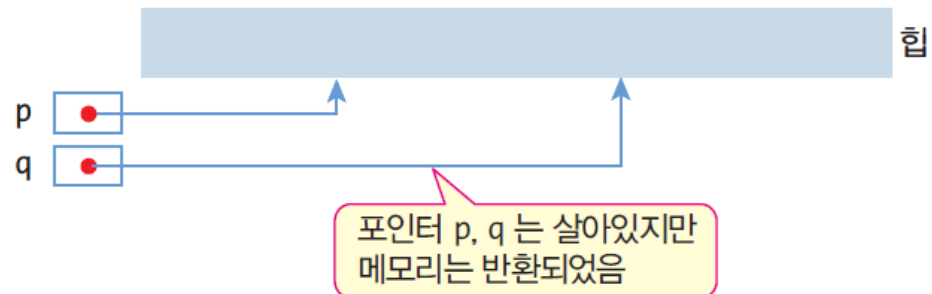
(1) `Circle *p = new Circle;`



(2) `Circle *q = new Circle(30);`



(3) `delete p;`
`delete q;`



예제 4-7 Circle 객체의 동적 생성 및 반환

22

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle();
    Circle(int r);
    ~Circle();
    void setRadius(int r) { radius = r; }
    double getArea() { return 3.14*radius*radius; }
};

Circle::Circle() {
    radius = 1;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::Circle(int r) {
    radius = r;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::~~Circle() {
    cout << "소멸자 실행 radius = " << radius << endl;
}
```

```
int main() {
    Circle *p, *q;
    p = new Circle;
    q = new Circle(30);
    cout << p->getArea() << endl << q->getArea() << endl;
    delete p;
    delete q;
}
```

생성한 순서에 관계 없이 원하는
순서대로 delete 할 수 있음

```
생성자 실행 radius = 1
생성자 실행 radius = 30
3.14
2826
소멸자 실행 radius = 1
소멸자 실행 radius = 30
```


예제 4-8 Circle 객체의 동적 생성과 반환 응용

23

정수 반지름을 입력 받고 Circle 객체를 동적 생성하여 면적을 출력하라. 음수가 입력되면 프로그램은 종료한다.

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle();
    Circle(int r);
    ~Circle();
    void setRadius(int r) { radius = r; }
    double getArea() { return 3.14*radius*radius; }
};

Circle::Circle() {
    radius = 1;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::Circle(int r) {
    radius = r;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::~~Circle() {
    cout << "소멸자 실행 radius = " << radius << endl;
}
```

```
int main() {
    int radius;
    while ( true ) {
        cout << "정수 반지름 입력(음수이면 종료)>> ";
        cin >> radius;
        if (radius < 0) break; // 음수가 입력되어 종료한다.
        Circle *p = new Circle(radius); // 동적 객체 생성
        cout << "원의 면적은 " << p->getArea() << endl;
        delete p; // 객체 반환
    }
}
```

delete 문이 없다면
메모리 누수 발생

정수 반지름 입력(음수이면 종료)>> 5
생성자 실행 radius = 5
원의 면적은 78.5
소멸자 실행 radius = 5
정수 반지름 입력(음수이면 종료)>> 9
생성자 실행 radius = 9
원의 면적은 254.34
소멸자 실행 radius = 9
정수 반지름 입력(음수이면 종료)>> -1

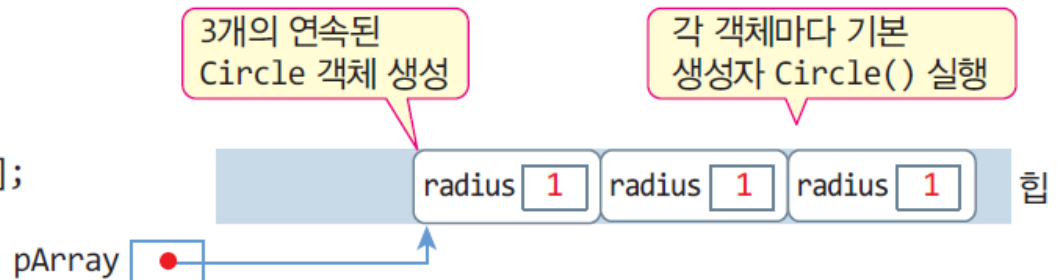
음수가 입력되면 종료

객체 배열의 동적 생성 및 반환

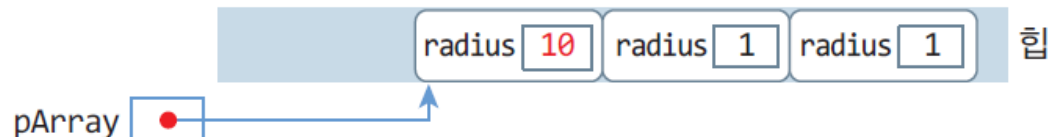
24

클래스이름 *포인터변수 = **new** 클래스이름 [배열 크기];
delete [] 포인터변수; // 포인터변수가 가리키는 객체 배열을 반환

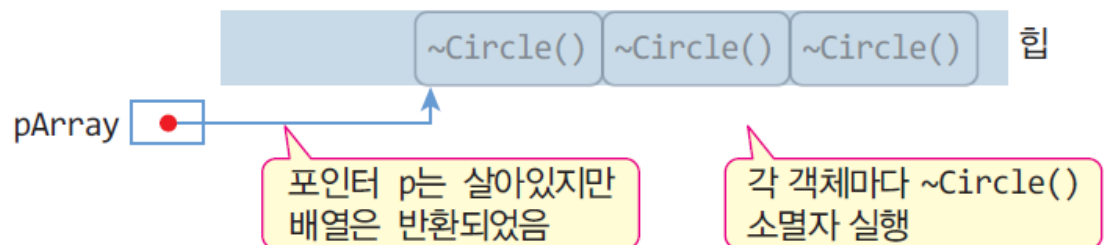
(1) Circle *pArray = new Circle[3];



(2) pArray[0].setRadius(10);



(3) delete [] pArray;



객체 배열의 사용, 배열의 반환과 소멸자

25

동적으로 생성된 배열도 보통 배열처럼 사용

```
Circle *pArray = new Circle[3]; // 3개의 Circle 객체 배열의 동적 생성
// Circle *pArray = new Circle[3] { Circle(10), Circle(20), Circle(30) }; // 각각의 원소 초기화

pArray[0].setRadius(10); // 배열의 첫 번째 객체의 setRadius() 멤버 함수 호출
pArray[1].setRadius(20); // 배열의 두 번째 객체의 setRadius() 멤버 함수 호출
pArray[2].setRadius(30); // 배열의 세 번째 객체의 setRadius() 멤버 함수 호출

for(int i=0; i<3; i++) {
    cout << pArray[i].getArea(); // 배열의 i 번째 객체의 getArea() 멤버 함수 호출
}
```

포인터로 배열 접근

```
for (Circle *p=pArray, *e=p+3; p < e; ++p)
    cout << p->getArea() << endl;
```

```
Circle *p = pArray;
for(int i=0; i<3; i++) {
    cout << p->getArea() << '\n';
    p++; // 다음 원소의 주소로 증가
}
```

배열 소멸

```
delete [] pArray;
```

pArray[2] 객체의 소멸자 실행(1)
pArray[1] 객체의 소멸자 실행(2)
pArray[0] 객체의 소멸자 실행(3)

각 원소 객체의 소멸자 별도 실행. 생성의 반대순

예제 4-9 Circle 배열의 동적 생성 및 반환

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle();
    Circle(int r);
    ~Circle();
    void setRadius(int r) { radius = r; }
    double getArea() { return 3.14*radius*radius; }
};

Circle::Circle() {
    radius = 1;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::Circle(int r) {
    radius = r;
    cout << "생성자 실행 radius = " << radius << endl;
}

Circle::~~Circle() {
    cout << "소멸자 실행 radius = " << radius << endl;
}
```

```
int main() {
    Circle *pArray = new Circle [3]; // 객체 배열 생성

    pArray[0].setRadius(10);
    pArray[1].setRadius(20);
    pArray[2].setRadius(30);

    for(int i=0; i<3; i++) {
        cout << pArray[i].getArea() << 'Wn';
    }

    Circle *p = pArray; // 포인터 p에 배열의 주소값으로 설정
    for(int i=0; i<3; i++) {
        cout << p->getArea() << 'Wn';
        p++; // 다음 원소의 주소로 증가
    }

    delete [] pArray; // 객체 배열 소멸
}
```

각 원소 객체의
기본 생성자 Circle() 실행

각 배열 원소 객체의
소멸자 ~Circle() 실행

생성자 실행 radius = 1
생성자 실행 radius = 1
생성자 실행 radius = 1
314
1256
2826
314
1256
2826
소멸자 실행 radius = 30
소멸자 실행 radius = 20
소멸자 실행 radius = 10

소멸자는 생성의
반대 순으로 실행

동적 할당이 아닌 배열의 포인터 사용하는
예제 4-6과 비교(강의노트 p.6)

예제 4-10 객체 배열의 동적 생성과 반환 응용

원을 개수를 입력 받고 Circle 배열을 동적 생성하라. 반지름 값을 입력 받아 Circle 배열에 저장하고, 면적이 100에서 200 사이인 원의 개수를 출력하라.

```
#include <iostream>
using namespace std;

class Circle {
    int radius;
public:
    Circle();
    ~Circle() {}
    void setRadius(int r) { radius = r; }
    double getArea() { return 3.14*radius*radius; }
};

Circle::Circle() {
    radius = 1;
}
```

생성하고자 하는 원의 개수?4

원1: 5

원2: 6

원3: 7

원4: 8

78.5 113.04 153.86 200.96

면적이 100에서 200 사이인 원의 개수는 2

```
int main() {
    cout << "생성하고자 하는 원의 개수?";
    int n, radius;
    cin >> n; // 원의 개수 입력

    Circle *pArray = new Circle [n]; // n 개의 Circle 배열 생성
    for(int i=0; i<n; i++) {
        cout << "원" << i+1 << ": "; // 프롬프트 출력
        cin >> radius; // 반지름 입력
        pArray[i].setRadius(radius); // 각 Circle 객체를 반지름으로 초기화
    }

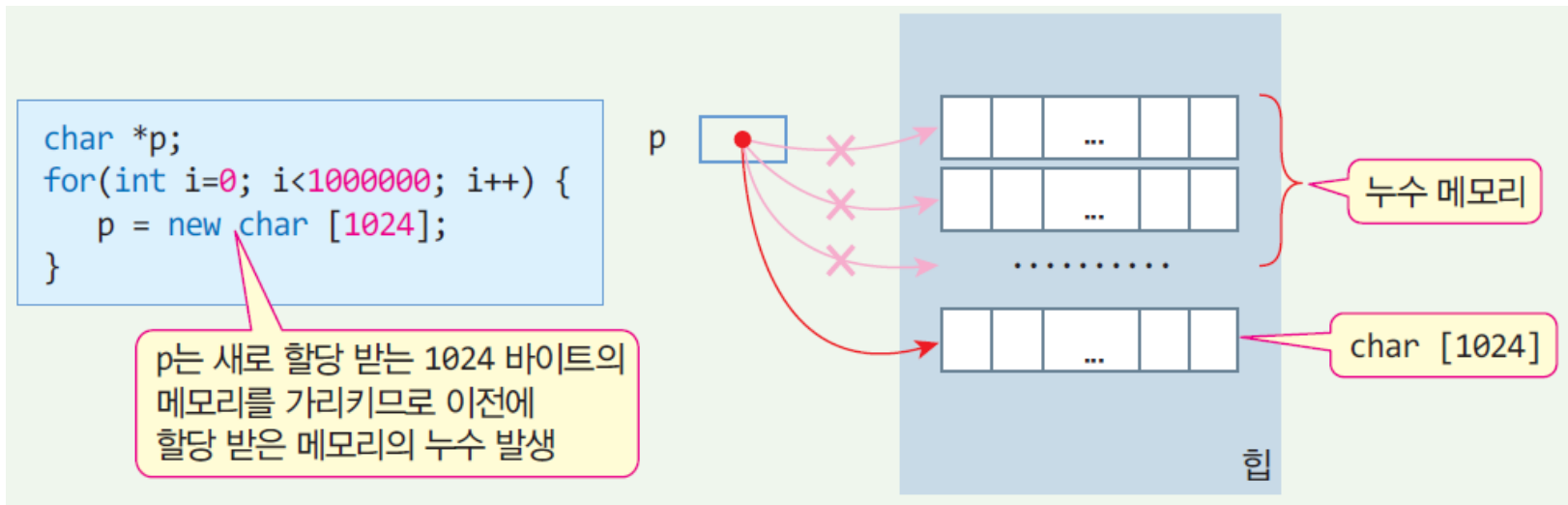
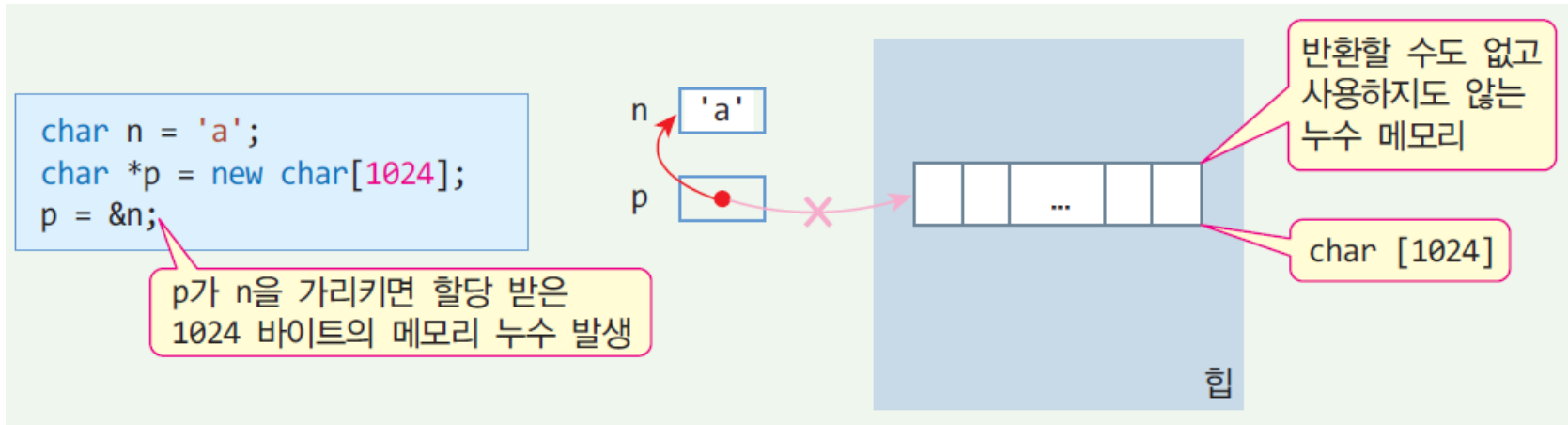
    int count =0; // 카운트 변수
    for (Circle* p = pArray, *e = p+n; p < e; ++p) {
        cout << p->getArea() << ' '; // 원의 면적 출력
        if (p->getArea() >= 100 && p->getArea() <= 200)
            count++;
    }

    cout << endl << "면적이 100에서 200 사이인 원의 개수는 "
        << count << endl;

    delete [] pArray; // 객체 배열 소멸
}
```

동적 메모리 할당과 메모리 누수

28



* 프로그램이 종료되면, 운영체제는 누수 메모리를 모두 힙에 반환

this 포인터

29

□ this

- 포인터, 객체 자신 포인터
- 클래스의 멤버 함수 내에서만 사용
- 개발자가 선언하는 변수가 아니고, 컴파일러가 선언한 변수
 - 멤버 함수에 컴파일러에 의해 묵시적으로 삽입 선언되는 매개 변수

```
class Circle {  
    int radius;  
public:  
    Circle() { radius=1; }  
    Circle(int r) { radius = r; }  
    void setRadius(int r) { radius = r; }  
    ....  
};
```

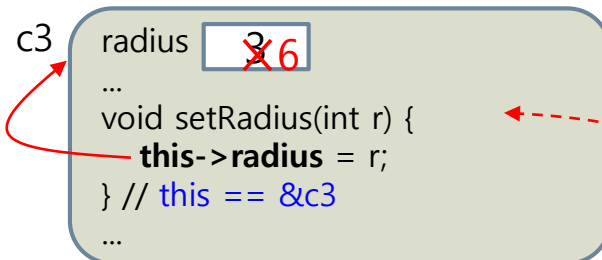
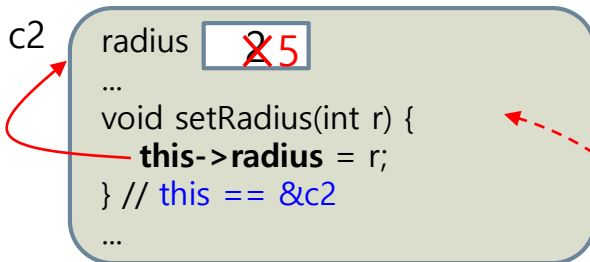
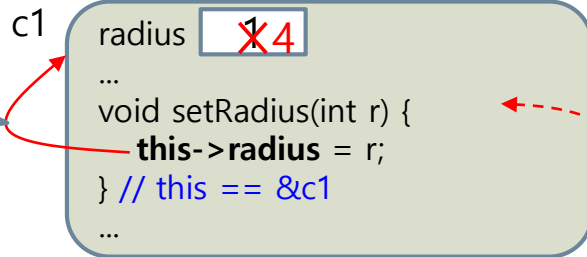
```
class Circle {  
    int radius;  
public:  
    Circle() { this->radius=1; }  
    Circle(int r) { this->radius = r; }  
    void setRadius(int r) { this->radius = r; }  
    ....  
};
```

this와 객체

30

* 각 객체 속의 this는 다른 객체의 this와 다름

this는 객체 자신
에 대한 포인터



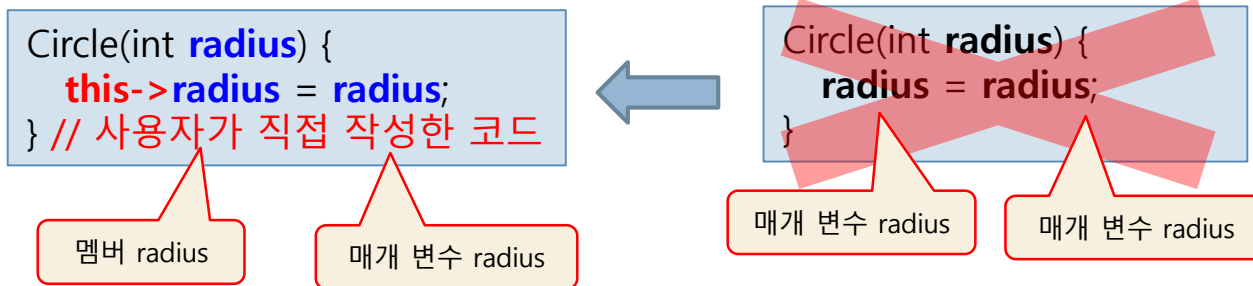
```
class Circle {  
    int radius;  
public:  
    Circle() {  
        this->radius=1;  
    }  
    Circle(int r) {  
        this->radius = r;  
    }  
    void setRadius(int r) {  
        this->radius = r;  
    }  
};
```

```
int main() {  
    Circle c1;  
    Circle c2(2);  
    Circle c3(3);  
  
    c1.setRadius(4);  
    c2.setRadius(5);  
    c3.setRadius(6);  
}
```


this가 필요한 경우

31

- 매개변수의 이름과 멤버 변수의 이름이 같은 경우



- 멤버 함수가 객체 자신의 주소를 리턴할 때
 - ▣ 연산자 중복 시에 매우 필요

```
class Sample {  
public:  
    Sample* f() {  
        ....  
        return this;  
    }  
};
```

```
// 사용 예  
Sample s;  
....  
Sample *p = s.f();  
// p == &s
```

this의 제약 사항

32

- 멤버 함수가 아닌 함수에서 this 사용 불가
 - ▣ 객체와의 관련성이 없기 때문
- static 멤버 함수에서 this 사용 불가
 - ▣ 객체가 생기기 전에 static 함수 호출이 있을 수 있기 때문에

this 포인터의 실체 – 컴파일러에서 처리

33

```
class Sample {  
    int a;  
public:  
    void setA(int x) {  
        this->a = x;  
    }  
};
```

(a) 개발자가 작성한 클래스

컴파일러에 의해
변환

```
class Sample {  
    ...  
public:  
    void setA(Sample* this, int x) {  
        this->a = x;  
    }  
};
```

this는 컴파일러에 의해 묵
시적으로 삽입된 매개 변수

(b) 컴파일러에 의해 변환된 클래스

Sample ob;

ob.setA(5);

컴파일러에 의해 변환

ob.setA(**&ob**, 5);

ob의 주소가 this 매개
변수에 전달됨

(c) 객체의 멤버 함수를 호출하는 코드의 변환

string 클래스를 이용한 문자열

34

□ C++ 문자열

- ▣ C-스트링
- ▣ C++ string 클래스의 객체

□ string 클래스

- ▣ C++ 표준 라이브러리, <string> 헤더 파일에 선언

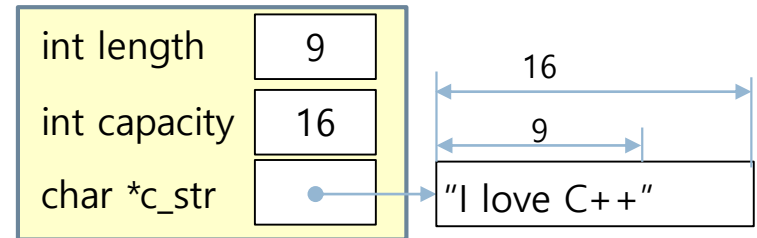
```
#include <string>
using namespace std;
```

▣ 가변 크기의 문자열

```
string str = "I love "; // str은 'I', ' ', 'I', 'o', 'v', 'e', ' '의 7개 문자로 구성
str.append("C++."); // str은 "I love C++."이 된다. 11개의 문자
```

- ▣ 다양한 문자열 연산을 실행하는 연산자와 멤버 함수 포함
 - 문자열 복사, 문자열 비교, 문자열 길이 등
- ▣ 문자열, 스트링, 문자열 객체, string 객체 등으로 혼용

string 객체

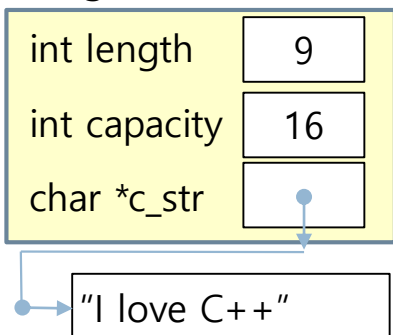


string 객체 생성 및 입출력

35

문자열 생성

string 객체



```
string str; // 빈 문자열을 가진 스트링 객체
string address("서울시 성북구 삼선동 389"); // 문자열 리터럴로 초기화
string address = "서울시 성북구 삼선동 389";
```

```
string copyAddress(address); // address를 복사한 copyAddress 생성
string copyAddress = address; // address를 복사한 copyAddress 생성
```

```
// C-스트링(char [] 배열)으로부터 스트링 객체 생성
```

```
char text[] = {'L', 'o', 'v', 'e', ' ', 'C', '+', '+', '\0'}; // Null 문자로 끝나는 문자 배열
```

```
char text[] = "Love C++"; // Null 문자로 끝나는 문자 배열
```

```
const char *text = "Love C++"; // Null 문자로 끝나는 문자 배열
```

```
string title(text); // "Love C++" 문자열을 가진 title 생성
```

```
string title(text, 4); // text에서 4글자, 즉 "Love" 문자열을 가진 title 생성
```

문자열 출력

■ cout과 << 연산자

```
cout << address << endl; // "서울시 성북구 삼선동 389" 출력
cout << title << endl; // "Love C++" 출력
```

문자열 입력

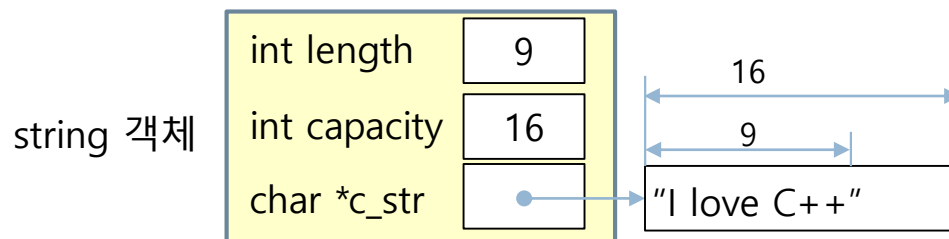
■ cin과 >> 연산자

```
string name;
cin >> name; // 공백이 입력되면 하나의 문자열로 입력
```

<표 4-2> string 클래스의 주요 멤버 함수

36

멤버 함수	멤버 함수
int size()	string& erase(int pos, int n)
int length()	string& append(string& str)
int capacity()	string& append(string& str, int pos, int n)
void clear()	string& replace(int pos, int n, string& str)
bool empty()	string& insert(int pos, string& str)
char *c_str()	int find(string& str), int find(char c)
char& at(int pos) s[pos]	int find(string& str, int pos) find(char c, pos)
int compare(string& str)	int rfind(string& str, int pos) rfind(char c, pos)
string substr(int pos, int n)	void swap(string& str1, string& str2)
string substr(int pos)	



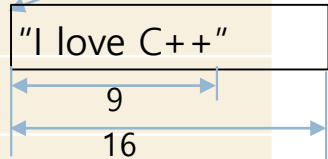
<표 4-2> string 클래스의 주요 멤버 함수

37

string 객체

[0][1][2][3]

string s = "C++", s1 = "C", s2 = "Java", s3;

int length	9
int capacity	16
char *c_str	

멤버 함수	멤버 함수 사용 예
int size()	int i = s.size() // i = 3
int length()	int i = s.length() // i = 3
int capacity()	int i = s.capacity() // i = 16
char *c_str()	char *p = s2.c_str() // p ----> "Java"
char& at(int pos) s[pos]	char c = s2.at(2) // == s2[2], c = 'v'
int compare(string& str)	int i = s1.compare(s2) // i = -1, i = s2.compare(s1) // i = 1 i = s.compare(s) // i = 0
string substr(int pos, int n)	s3 = s2.substr(1, 2) // s3 = "av"
string substr(int pos)	s3 = s2.substr(1) // s3 = "ava"
void clear()	s.clear() // s = ""
bool empty()	bool b = s.empty() // b = false

<표 4-2> string 클래스의 주요 멤버 함수

38

0123456789

0123

```
string s = "C++", s1 = "C Language", s2 = "Java", s3;
```

멤버 함수 사용 예

```
s1.erase(3, 7) == s1.erase(3) // s1 = "C L";
```

```
s1.erase()      == s1.clear()  // s1 = ""
```

```
s.append(s2)      // s = "C++Java", s += s2와 동일한 연산
```

```
s.append(s2, 1, 2) // s = "C++av"
```

```
s1.insert(2, "++ ") // s1 = "C ++ Language"
```

```
s1.replace(2, 8, "Programming") // s1 = "C Programming"
```

0123456789

```
string e = "I love love C++";
```

```
int index = e.find("love"); index = e.find('l'); // index = 2
```

```
index = e.find("love", index+1); // index = 7
```

```
index = e.find("C#"); index = e.find('#'); // index = -1, 못 찾았을 경우
```

```
index = e.find('v', 7); // index = 9
```

```
index = e.rfind("love"); index = e.rfind('l'); // index = 7
```

```
index = e.rfind("love", 6); index = e.rfind('l', 6); // index = 2
```

```
swap(s1, s2); // s1 = "Java", s2 = "C Language"
```


<표 4-3> string 클래스의 연산자

39

```
string s = "C++", s1 = "C", s2 = "Java", s3;
```

연산자 사용 예	결과
s1 = s2	s1 = "Java"
char c = s[1]	c = '+'
s3 = s1 + s2	s3 = "CJava"
s1 += s2	s1 = "CJava"
s1 == s2	false
s1 != s2	true
s1 < s2	true
s1 > s2	false
s1 <= s2	true
s1 >= s2	false

string 객체의 동적 생성

40

```
float n = stof("34.1");  
double n = stod("34.1");  
string s = to_string(10); // integer  
string s = to_string(10.5); // double
```

□ 문자열 숫자 변환

- stoi() 함수 이용
 - 2011 C++ 표준부터
- 비주얼 C++ 2008은 2011 표준 지원않음

```
string s="123";  
int n = stoi(s); // n은 정수 123. 비주얼 C++ 2010 이상 버전
```

```
string s="123";  
int n = atoi(s.c_str()); // n은 정수 123. 비주얼 C++ 2008 이하
```

□ new/delete를 이용하여 문자열을 동적 생성/반환 가능

```
string *p = new string("C++"); // 스트링 객체 동적 생성  
  
cout << *p; // "C++" 출력  
p->append(" Great!!"); // p가 가리키는 스트링이 "C++ Great!!"이 됨  
cout << *p; // "C++ Great!!" 출력  
  
delete p; // 스트링 객체 반환
```

□ 표준 클래스 및 멤버 함수 검색

- <http://www.cplusplus.com/reference/> 사이트의
- Search: [] 창에서 검색할 클래스 또는 멤버 함수 입력

예제 4-11 string 클래스를 이용한 문자열 생성 및 출력

41

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // 스트링 생성
    string str; // 빈 문자열을 가진 스트링 객체 생성
    string address("서울시 성북구 삼선동 389");
    string copyAddress(address); // address의 문자열을 복사한 스트링 객체 생성

    char text[] = {'L', 'o', 'v', 'e', ' ', 'C', '+', '+', '\0'}; // C-스트링
    string title(text); // "Love C++" 문자열을 가진 스트링 객체 생성

    // 스트링 출력
    cout << str << endl; // 빈 스트링. 아무 값도 출력되지 않음
    cout << address << endl;
    cout << copyAddress << endl;
    cout << title << endl;
}
```

string 클래스
를 사용하기 위
해 반드시 필요

빈 문자열을 가
진 스트링 출력

서울시 성북구 삼선동 389
서울시 성북구 삼선동 389
Love C++

예제 4-12 string 배열 선언과 문자열 키 입력 응용

42

5 개의 string 배열을 선언하고

getline()을 이용하여 문자열을 입력 받아

사전 순으로 가장 뒤에 나오는 문자열을 출력하라.

문자열 비교는 <, > 연산자를 간단히 이용하면 된다.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string names[5]; // 문자열 배열 선언

    for(int i=0; i<5; i++) {
        cout << "이름 >> ";
        getline(cin, names[i], 'Wn'); // getline(cin, names[i])과 동일
    }

    string latter = names[0];
    for(int i=1; i<5; i++) {
        if (latter < names[i]) { // 사전 순으로 latter 문자열이 앞에 온다면
            latter = names[i]; // latter 문자열 변경
        }
    }
    cout << "사전에서 가장 뒤에 나오는 문자열은 " << latter << endl;
}
```

```
이름 >> Kim Nam Yun
이름 >> Chang Jae Young
이름 >> Lee Jae Moon
이름 >> Han Won Sun
이름 >> Hwang Su hee
사전에서 가장 뒤에 나오는 문자열은 Lee Jae Moon
```

예제 4-13 문자열을 입력 받고 회전시키기

43

빈칸을 포함하는 문자열을 입력 받고, 한 문자씩 왼쪽으로 회전하도록 문자열을 변경하고 출력하라.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s;

    cout << "문자열을 입력하세요(한글 안됨) " << endl;
    getline(cin, s, '\n'); // 문자열 입력; getline(cin, s); 과 동일
    int len = s.length(); // 문자열의 길이

    for(int i=0; i<len; i++) {
        string first = s.substr(0,1); // 맨 앞의 문자 1개를 문자열로 분리
        string sub = s.substr(1, len-1); // 나머지 문자들을 문자열로 분리
        // s.substr(1, len-1)는 s.substr(1) 과 동일
        s = sub + first; // 두 문자열을 연결하여 새로운 문자열로 만들
        cout << s << endl;
    }
}
```

문자열을 입력하세요 (한글 안됨)

I love you

love youl

love youl

ove youl I

ve youl lo

e youl lov

youl love

youl love

oul love y

ul love yo

I love you

예제 4-14 문자열 처리 응용 - 덧셈 문자열을 입력 받아 덧셈 실행

4+125+4+77+102 등으로 표현된 덧셈식을 문자열로 입력 받아 계산하는 프로그램 작성하라.

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {
    string s;
    cout << "7+23+5+100+25와 같이 덧셈 문자열을 입력하세요." << endl;
```

```
    getline(cin, s, '\n'); // 문자열 입력; getline(cin, s); 과 동일
```

```
    int sum = 0;
```

```
    int startIndex = 0; // 문자열 내에 검색할 시작 인덱스
```

```
    while (true) {
```

```
        int flIndex = s.find('+', startIndex); if (flIndex == string::npos)
```

```
        if (flIndex == -1) { // '+' 문자 발견할 수 없음
```

```
            string part = s.substr(startIndex); // 끝까지
```

```
            if (part == "") break; // "2+3+", 즉 +로 끝나는 경우
```

```
            cout << part << endl;
```

```
            sum += stoi(part); // 문자열을 수로 변환하여 더하기
            break;
        }
```

```
        int count = flIndex - startIndex; // 서브스트링으로 자른 문자 개수
```

```
        string part = s.substr(startIndex, count); // startIndex부터 count 개의 문자로 서브스트링 만들기
```

```
        cout << part << endl;
```

```
        sum += stoi(part); // 문자열을 수로 변환하여 더하기
```

```
        startIndex = flIndex+1; // 검색을 시작할 인덱스 전진시킴
```

```
    }
```

```
    cout << "숫자들의 합은 " << sum;
```

```
}
```

7+23+5+100+25와 같이 덧셈 문자열을 입력하세요.

66+2+8+55+100

66

2

8

55

100

숫자들의 합은 231

0123456789012
66+2+8+55+100

예제 4-15 문자열 find 및 replace

&가 입력될 때까지 여러 줄의 영문 문자열을 입력 받고, 찾는 문자열과 대체할 문자열을 각각 입력 받아 문자열을 변경하라.

여러 줄의 문자열을 입력하세요. 입력의 끝은 &문자입니다.

It's **now** or never, come hold me tight. Kiss me my darling, be mine tonight
Tomorrow will be too late. It's **now** or never, my love won't wait&

find: **now**

검색할 단어

replace: **Right Now**

대체할 단어

& 뒤에 <Enter>키 입력

It's **Right Now** or never, come hold me tight. Kiss me my darling, be mine tonight
Tomorrow will be too late. It's **Right Now** or never, my love won't wait

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {
    string s;
    cout << "여러 줄의 문자열을 입력하세요. 입력의 끝은 &문자입니다." << endl;
    getline(cin, s, '&'); // 문자열 입력
    cin.ignore();
    string f, r;
    cout << endl << "find: ";
    getline(cin, f, '\n'); // 검색할 문자열 입력
    cout << "replace: ";
    getline(cin, r, '\n'); // 대체할 문자열 입력

    int startIndex = 0;
    while(true) {
        int fIndex = s.find(f, startIndex); // startIndex부터 문자열 f 검색
        if(fIndex == -1)
            break; // 문자열 s의 끝까지 변경하였음
        s.replace(fIndex, f.length(), r); // fIndex부터 문자열 f의 길이만큼 문자열 r로 변경
        startIndex = fIndex + r.length();
    }
    cout << s << endl;
}
```

& 뒤에 따라 오는 <Enter> 키를 제거하기 위한 코드!!!

if (fIndex == **string::npos**)